

VX-01

Docker Infrastructure — User Manual
Hybrid Amphibious Search & Rescue Robot

ROS2 Humble · Ignition Fortress 6 · Raspberry Pi 5 ARM64 · Laptop AMD64
v2.0 — github.com/Gouri-Shankar-85/VX-01

Part 1 — Project Structure

Your repository lives at `~/vx-01/` on your laptop. This is the full layout after all files are in place:

```
~/vx-01/
├── .env ← DOCKER_REGISTRY=gourishankar85
├── .dockerignore
├── build.sh ← all build commands
├── deploy-rdk.sh ← Pi 5 first-time setup
├── docker-compose.yml ← laptop profiles
├── docker-compose.rdk.yml ← Pi 5 containers
├── docker/
│   ├── Dockerfile.base ← ROS2 Humble hardware image
│   ├── Dockerfile.sim ← extends base + Gazebo/RViz
│   └── Dockerfile.dashboard ← React dashboard image
├── entrypoint.sh
├── udev/
├── 99-vx01-devices.rules
├── vx01_dashboard/
├── frontend/ ← React + Vite + Tailwind
├── vx01_ws/
└── src/ ← your ROS2 packages
```

Part 2 — Containers Overview

On the **laptop** (`docker-compose.yml`), containers are started with profiles:

Container	Profile	Image	What it does
vx01-dev	dev	vx01-sim:humble	Interactive shell with all tools + sim
vx01-hardware	hardware	vx01-base:humble	Real sensors on laptop
vx01-sim	sim	vx01-sim:humble	Gazebo Fortress simulation
vx01-rosbridge	dev/hw/sim	vx01-base:humble	WebSocket bridge for dashboard (port 9090)
vx01-dashboard	dev/hw/sim	vx01-dashboard:latest	React dashboard served on port 5173

On the **Raspberry Pi 5** (`docker-compose.rdk.yml`), all containers run always without profiles:

Container	Image	What it does
vx01-robot	vx01-base:humble-arm64	Main robot container — idle, enter with <code>exec</code>
vx01-mavros	vx01-base:humble-arm64	ArduPilot bridge — idle, start MAVROS inside
vx01-rosbridge	vx01-base:humble-arm64	rosbridge WebSocket on port 9090
vx01-dashboard	vx01-dashboard:latest	Dashboard served on port 5173

Part 3 — One-Time Laptop Setup

3.1 Requirements

Ubuntu 22.04, Docker Engine 24+, docker buildx. Check:

```
docker --version
```

```
docker compose version
```

```
docker buildx version
```

If Docker is missing:

```
curl -fsSL https://get.docker.com | sudo sh
```

```
sudo usermod -aG docker $USER
```

```
# Log out and back in after this
```

3.2 Install udev Device Rules

Run once on your laptop host. Creates stable symlinks for all VX-01 devices:

```
cd ~/vx-01
```

```
chmod +x build.sh
```

```
./build.sh udev-install
```

```
# Log out and back in after this
```

3.3 Allow Docker to Open GUI Windows

Add to **~/.bashrc** so it runs on every login:

```
echo 'xhost +local:docker' >> ~/.bashrc
```

```
source ~/.bashrc
```

3.4 SSH for GitHub

No setup needed. The compose files mount your **~/.ssh** and **~/.gitconfig** read-only into every container. Git push works inside any container automatically using your host SSH key.

Part 4 — Building Images

4.1 Build the Hardware Image (AMD64)

```
cd ~/vx-01
./build.sh build-base
# First build: 20-40 min. Subsequent builds are fast (Docker cache).
```

4.2 Build the Simulation Image (AMD64 only)

```
./build.sh build-sim
```

4.3 Build the Dashboard Image (AMD64 → push to Docker Hub)

```
./build.sh build-dashboard
```

■ This builds the React frontend, packages it into a Node image, and pushes to Docker Hub. The Pi pulls this image — it runs natively on ARM64.

4.4 Build ARM64 Image for Raspberry Pi 5

Recommended — uses GitHub Actions (native ARM64 runner, no QEMU):

```
./build.sh build-arm-ci
# Triggers .github/workflows/build-arm.yml
# Monitor: https://github.com/Gouri-Shankar-85/VX-01/actions
# Wait for green before deploying to Pi.
```

Alternative — local cross-compile via QEMU (may be slow or fail):

```
./build.sh build-arm
```

4.5 Push All AMD64 Images to Docker Hub

```
./build.sh push
```

4.6 When to Rebuild

Situation	Rebuild?	Command
Changed .cpp or .py in vx01_ws/src/	No	colcon build inside container
Added a new ROS2 package to src/	No	colcon build inside container
Added an apt package to Dockerfile.base	Yes	./build.sh build-base then build-arm-ci
Changed entrypoint.sh	Yes	./build.sh build-base then build-arm-ci
Changed the dashboard (React files)	Yes	./build.sh build-dashboard
Nothing changed, starting work today	No	just start containers

Part 5 — Daily Laptop Workflow

5.1 Start Dev Container

```
cd ~/vx-01
```

```
docker compose --profile dev up -d
```

```
docker exec -it vx01-dev bash
```

■ Your src/ at ~/vx-01/vx01_ws/src/ is mounted live. Edit on laptop, build inside container.

5.2 Build ROS2 Workspace (inside container)

```
cd /vx01_ws
```

```
colcon build --symlink-install
```

```
source install/setup.bash
```

5.3 Run Simulation

```
# Start sim profile (includes rosbridge + dashboard):
```

```
docker compose --profile sim up -d
```

```
# Or launch Gazebo from inside a running dev container:
```

```
ros2 launch vx01_simulation sim.launch.py
```

5.4 Run Real Hardware on Laptop

```
docker compose --profile hardware up -d
```

```
docker exec -it vx01-hardware bash
```

5.5 Git Push from Inside Container

```
cd /vx01_ws
```

```
git add src/your_package/
```

```
git commit -m "your message"
```

```
git push
```

```
# SSH key from host ~/.ssh is used automatically
```

5.6 Stop Everything

```
docker compose --profile dev down
```

```
docker compose --profile sim down
```

```
docker compose --profile hardware down
```

Part 6 — Raspberry Pi 5 Setup

6.1 First-Time Setup (run once)

Copy the deploy script to the Pi and run it. The Pi user is **pi**, host OS is Ubuntu 24.04:

```
# On your laptop:
scp deploy-rdk.sh pi@:/home/pi/
scp -r docker/udev pi@:/home/pi/docker/
```

```
# SSH into the Pi:
ssh pi@
```

```
# On the Pi:
chmod +x deploy-rdk.sh
./deploy-rdk.sh gourishankar85
```

The script automatically:

- Installs Docker if missing
- Installs docker-compose plugin
- Installs udev rules for all VX-01 devices
- Creates ~/vx01/docker-compose.rdk.yml
- Pulls gourishankar85/vx01-base:humble-arm64
- Pulls gourishankar85/vx01-dashboard:latest
- Starts all 4 containers with restart: unless-stopped

■ After deploy-rdk.sh finishes, all containers start automatically on every Pi reboot.

6.2 Access Dashboard

```
# Open in browser on any device on the same network:
http://:5173
```

```
# The dashboard connects to rosbridge automatically
# using the same IP the page was opened from.
# No static IP or config needed.
```

6.3 Daily Pi Commands

What	Command
Check all containers running	<code>docker ps</code>
Open robot shell	<code>docker exec -it vx01-robot bash</code>
See robot logs	<code>docker logs -f vx01-robot</code>
See rosbridge logs	<code>docker logs -f vx01-rosbridge</code>
Check devices connected	<code>ls -la /dev/ttyPIXHAWK /dev/ttyMAESTRO /dev/ttyTFMINI /dev/ttyIMU</code>
Restart one container	<code>docker restart vx01-robot</code>
Stop all containers	<code>docker compose -f ~/vx01/docker-compose.rdk.yml down</code>

Start all containers	<code>docker compose -f ~/vx01/docker-compose.rdk.yml up -d</code>
See CPU/RAM usage	<code>docker stats</code>

6.4 Update Pi After Image Change

```
# On your laptop – rebuild and push (example for base image):
```

```
./build.sh build-arm-ci
```

```
# Wait for GitHub Actions to finish (green checkmark)
```

```
# On the Pi:
```

```
docker pull gourishankar85/vx01-base:humble-arm64
```

```
docker compose -f ~/vx01/docker-compose.rdk.yml down
```

```
docker compose -f ~/vx01/docker-compose.rdk.yml up -d
```

For dashboard updates only (faster — no ARM build needed):

```
# On your laptop:
```

```
./build.sh build-dashboard
```

```
# On the Pi:
```

```
docker pull gourishankar85/vx01-dashboard:latest
```

```
docker restart vx01-dashboard
```

Part 7 — Devices and Stable Port Names

udev rules create fixed symlinks based on USB vendor/product ID. Plug order and USB port do not matter.

Device	Symlink	Protocol	Baud
Radilink PIX6 (ArduPilot)	/dev/ttyPIXHAWK	USB ACM	921600
Pololu Maestro 18ch	/dev/ttyMAESTRO	USB HID + Serial	—
Benewake TFmini-S LiDAR	/dev/ttyTFMINI	UART	115200
Hiwonder IM10A IMU	/dev/ttyIMU	UART	9600
YDLIDAR HP60C Camera	/dev/video0	USB UVC	—

To find the vendor ID of an unknown device:

```
lsusb
```

```
# Example output: ID 10c4:ea60 Silicon Labs CP210x
```

```
# Format is vendor:product
```

```
udevadm info -a -n /dev/ttyUSB0
```

```
# Shows all attributes – use ATTRS{idVendor} and ATTRS{idProduct}
```

Part 8 — Git Workflow

8.1 Normal Code Change (no Docker rebuild needed)

```
# On laptop – edit files in ~/vx-01/vx01_ws/src/
```

```
# Start container if not running:
```

```
cd ~/vx-01
```

```
docker compose --profile dev up -d
```

```
docker exec -it vx01-dev bash
```

```
# Inside container:
```

```
cd /vx01_ws
```

```
colcon build --symlink-install
```

```
source install/setup.bash
```

```
# Push to GitHub:
```

```
git add src/your_package/
```

```
git commit -m "your change"
```

```
git push
```

8.2 Dockerfile or entrypoint.sh Changed

```
# On laptop:
```

```
git add docker/
```

```
git commit -m "update dockerfile"
```

```
git push
```

```
# Rebuild base image and trigger ARM64 build:
```

```
./build.sh build-base
```

```
./build.sh build-arm-ci
```

```
# Wait for GitHub Actions green
```

```
# Then update Pi:
```

```
docker pull gourishankar85/vx01-base:humble-arm64
```

```
docker compose -f ~/vx01/docker-compose.rdk.yml down
```

```
docker compose -f ~/vx01/docker-compose.rdk.yml up -d
```

8.3 Dashboard Changed

```
# On laptop:
```

```
git add vx01_dashboard/
```

```
git commit -m "update dashboard"
```

```
git push
```

```
./build.sh build-dashboard
```

```
# On Pi:
```

```
docker pull gourishankar85/vx01-dashboard:latest
```

```
docker restart vx01-dashboard
```

Part 9 — Troubleshooting

Problem	Cause	Fix
exec format error on Pi	Wrong arch image (AMD64 on ARM64)	Wait for build-arm-ci to finish, then pull again
Container keeps restarting on Pi	entrypoint.sh has Windows line endings (CRLF)	On laptop: dos2unix docker/entrypoint.sh, rebuild, push
/dev/ttyTFMINI not found	udev rules not installed	./build.sh udev-install on host, replug device
Permission denied on /dev/tty*	Not in dialout group	sudo usermod -aG dialout \$USER, log out and back in
MAVROS connected: False	Wrong device or baud	Check ls /dev/ttyPIXHAWK exists, baud must be 921600
Dashboard shows ROS Disconnected	rosbridge not running or wrong port	docker logs -f vx01-rosbridge — check port 9090
Gazebo/RViz window not opening	X11 not allowed	xhost +local:docker on host, restart container
colcon build: missing package	rosdep not run	rosdep install --from-paths src --ignore-src -r -y
docker buildx build fails	No buildx builder created	docker buildx create --name vx01-builder --use
Two CP2102 devices, same symlink	udev rule collision	udevadm info -a -n /dev/ttyUSB0 — use ATTRS{serial}

Part 10 — Quick Reference

One-Time Laptop Setup

```
cd ~/vx-01
./build.sh udev-install
echo 'xhost +local:docker' >> ~/.bashrc && source ~/.bashrc
./build.sh build-base
./build.sh build-sim
./build.sh build-dashboard
./build.sh build-arm-ci # trigger GitHub Actions ARM64 build
./build.sh push # push AMD64 images
```

Every Day — Laptop

```
cd ~/vx-01
docker compose --profile dev up -d
docker exec -it vx01-dev bash
# inside container:
cd /vx01_ws && colcon build --symlink-install
git pull / git push
docker compose --profile dev down
```

Pi 5 — First Time

```
# On laptop:
scp deploy-rdk.sh pi@:/home/pi/
scp -r docker/udev pi@:/home/pi/docker/

# On Pi:
chmod +x ~/deploy-rdk.sh
~/deploy-rdk.sh gourishankar85
```

Pi 5 — After Any Image Update

```
# Base image update:
docker pull gourishankar85/vx01-base:humble-arm64
docker compose -f ~/vx01/docker-compose.rdk.yml down
docker compose -f ~/vx01/docker-compose.rdk.yml up -d

# Dashboard update only:
docker pull gourishankar85/vx01-dashboard:latest
docker restart vx01-dashboard
```

Useful URLs (on same network as Pi)

```
Dashboard: http://:5173
Rosbridge: ws://:9090
```